Mojo🔥

Mojo metaprogramming magic

One C ABI, three problems

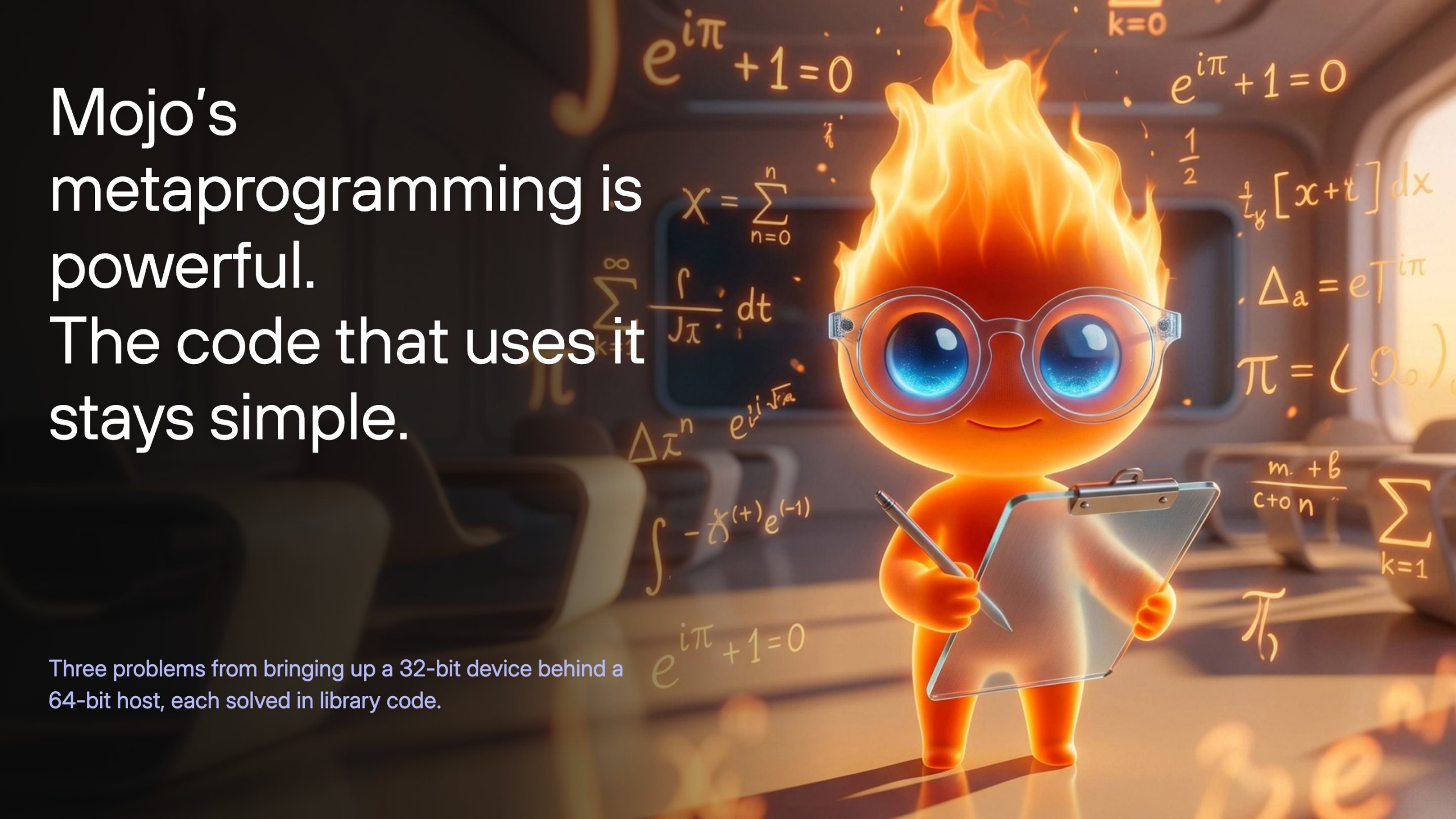
Kern Handa

October 26, 2026

Modular



Mojo's metaprogramming is powerful. The code that uses it stays simple.



Three problems from bringing up a 32-bit device behind a 64-bit host, each solved in library code.

The north star

Bringing up a new accelerator *should* be mostly library work. The compiler lowers and links. Mojo code does the rest.

COMPILER SIDE

Lowers and links

Each backend's compiler does lowering and linking, and reads the attributes the library attaches.

MOJO LIBRARY

Everything in between

From ctx.enqueue_function(...) to the device entry point: argument marshalling, the generated entry wrapper, closure transport and launch metadata.

KERNEL AUTHORS

Same code as on GPUs

They write the same typed compile_function / enqueue_function code they write for CUDA, HIP or Metal.

WHAT A KERNEL LOOKS LIKE

One kernel, two sides

device

```
def vector_add(
    lhs: Pointer[Float32, MutAnyOrigin],
    rhs: Pointer[Float32, MutAnyOrigin],
    result: Pointer[Float32, MutAnyOrigin],
    n: Int,                                  # its real signature
):
    var i = global_idx.x
    if i < n:
        result[unsafe_offset=i] = lhs[unsafe_offset=i] + rhs[unsafe_offset=i]
```

host

```
def main():
    var ctx = DeviceContext()
    var lhs = ctx.enqueue_create_buffer[DType.float32](n)
    ...                                      # rhs, result likewise
    var kernel = ctx.compile_function[vector_add]()
    ctx.enqueue_function(
        kernel, lhs, rhs, result, n,
        grid_dim=ceildiv(n, 256), block_dim=256,
    )
    ctx.synchronize()
```

Device: a plain typed function. Its parameters are the kernel's real signature.

Host: a DeviceContext allocates buffers, compiles the kernel and enqueues the launch.

Everything between **enqueue_function** and the device entry point is library code.

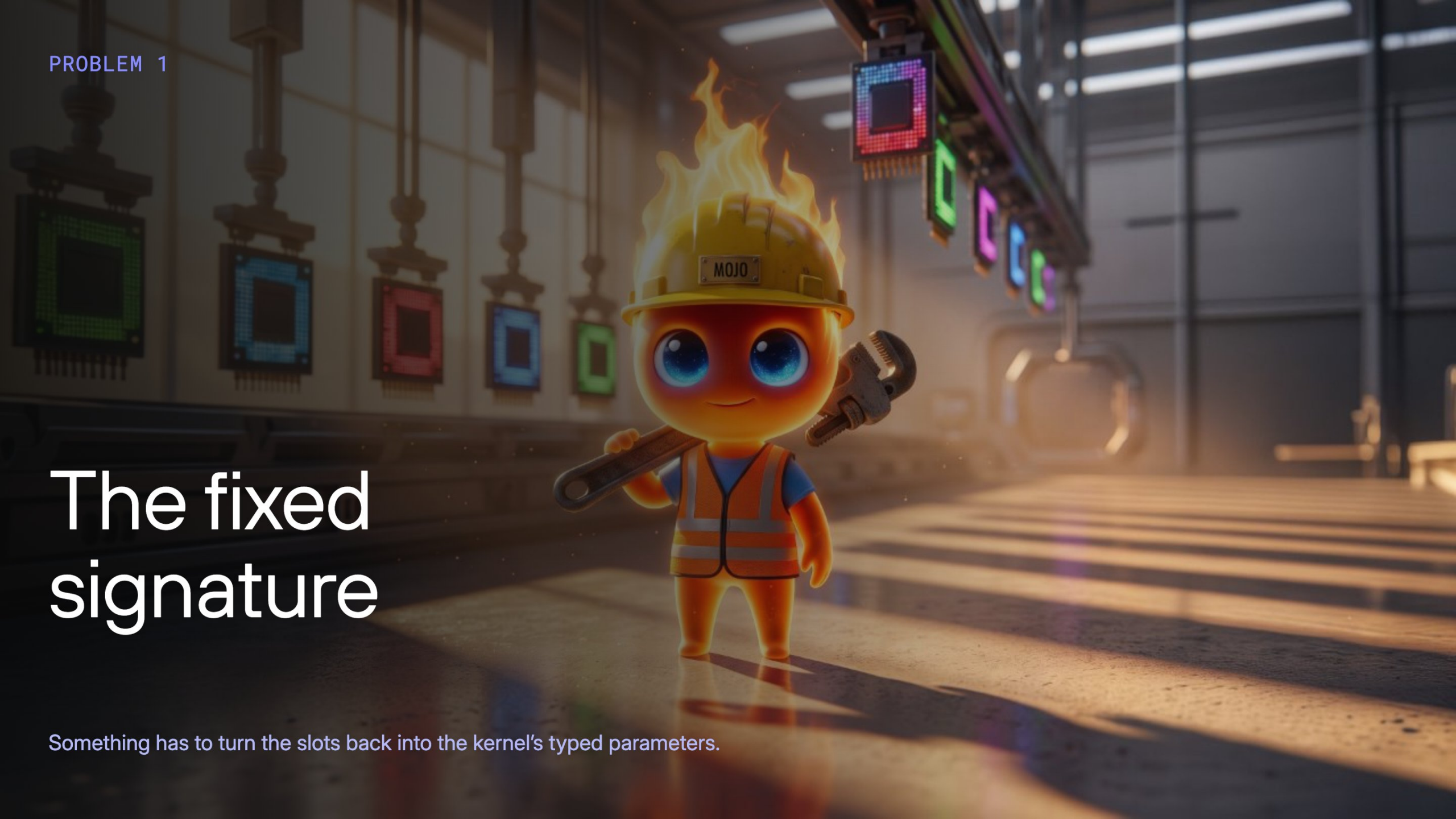
AGENDA

- The fixed FFI signature 01
- FAM packing of the arguments 02
- Closure serialization and pointer patching 03

Each problem gets three passes

By hand	Generated	What kernel authors write
C++ and Mojo, no metaprogramming	C++ and Mojo, where C++ can	the result, from their side

PROBLEM 1



The fixed signature

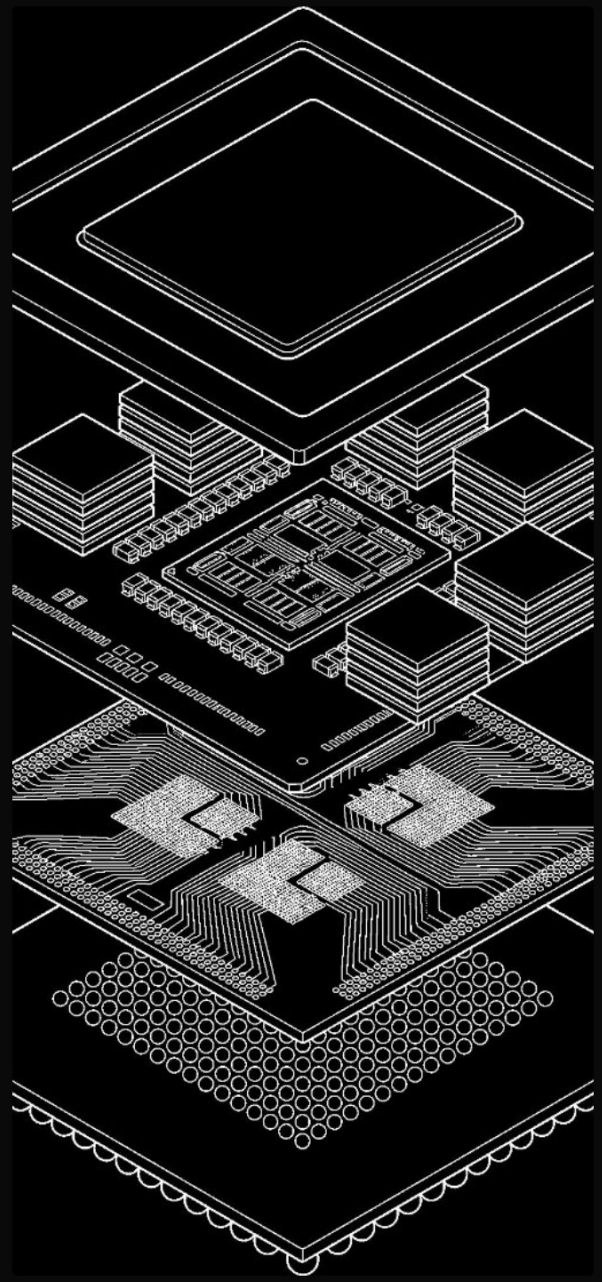
Something has to turn the slots back into the kernel's typed parameters.

THE SETUP

One C entry point for every kernel

```
1  struct LaunchInfo;  /* opaque: which core and thread this invocation runs on */
2
3  struct ArgArray {
4    uint64_t count;     /* number of slots */
5    uint64_t slots[];   /* C99 flexible array member (FAM) */
6  };
7
8  uint32_t entry(const LaunchInfo *info, const ArgArray *args);
```

The device runtime calls every kernel through it. Arguments arrive as an array of device addresses.



By hand, in C++

```cpp
template <class T>
T* slot(const ArgArray* a, std::size_t i) {
  return reinterpret_cast<T*>(static_cast<uintptr_t>(a->slots[i]));
}

extern "C" uint32_t vector_add_entry(const LaunchInfo* info, const ArgArray* args) {
  const float* lhs = slot<const float>(args, 0);
  const float* rhs = slot<const float>(args, 1);
  float* result = slot<float>(args, 2);
  int32_t n = *slot<const int32_t>(args, 3);
  auto [start, end] = work_range(info, n);
  for (int32_t i = start; i < end; ++i) result[i] = lhs[i] + rhs[i];
  return 0;
}
```

Cast each slot back to a typed pointer, by index.

Slot 3 holds the address of n, not n itself.

device

```
def get_slot[T: AnyType](
    args: Pointer[ArgArray, MutAnyOrigin], i: Int
) -> Pointer[T, MutAnyOrigin]:
    var words = args.unsafe_bitcast[UInt64]()  # word 0 is the count
    return Pointer[T, MutAnyOrigin](
        unsafe_from_address=Int(words[unsafe_offset=i + 1])
    )


def vector_add_entry(
    info: Pointer[LaunchInfo, MutAnyOrigin],
    args: Pointer[ArgArray, MutAnyOrigin],
) -> UInt32:
    var lhs = get_slot[Float32](args, 0)
    var rhs = get_slot[Float32](args, 1)
    var result = get_slot[Float32](args, 2)
    var n = Int(get_slot[Int32](args, 3)[])
    var (start, end) = work_range(n)
    for i in range(start, end):
        result[unsafe_offset=i] = lhs[unsafe_offset=i] + rhs[unsafe_offset=i]
    return 0
```

1.1 · BY HAND · MOJO

By hand, in Mojo

With no metaprogramming, Mojo reads like C. No runtime and no magic between you and the ABI.

Both versions leave the same gap: parameters and slot indices are kept in sync by hand.

The kernel we're after: just a typed function

device

```
def vector_add(
    lhs: Pointer[Float32, MutAnyOrigin],
    rhs: Pointer[Float32, MutAnyOrigin],
    result: Pointer[Float32, MutAnyOrigin],
    n: Int,
):
    var i = global_idx.x
    if i < n:
        result[unsafe_offset=i] = lhs[unsafe_offset=i] + rhs[unsafe_offset=i]
```

1. Its parameters are the real signature: three pointers and an Int.
2. No ArgArray, no get_slot, no slot indices.
3. Problem 1: generate the entry wrapper from this signature.

what the previous slide wired up by hand

slot 0 → lhs	slot 1 → rhs	slot 2 → result	slot 3 → n

Write the function. Let the library generate the mapping.

```cpp
// The kernel's parameter types, from its signature.
template <class F> struct fn_traits;
template <class... A> struct fn_traits<void (*)(A...)> { using args = std::tuple<A...>; };

template <class Args, std::size_t I>
constexpr std::size_t slot_index() {  // slots used by parameters 0..I-1
  return []<std::size_t... J>(std::index_sequence<J...>) {
    return (std::size_t{0} + ... + slot_count<std::tuple_element_t<J, Args>>());
  }(std::make_index_sequence<I>{});
}

template <auto Kernel>
uint32_t entry(const LaunchInfo*, const ArgArray* a) {
  using Args = typename fn_traits<decltype(Kernel)>::args;
  [&]<std::size_t... I>(std::index_sequence<I...>) {
    // Function arguments are unsequenced, so each slot index is computed up
    // front rather than with a running counter.
    Kernel(load_arg<std::tuple_element_t<I, Args>>(a, slot_index<Args, I>())...);
  }(std::make_index_sequence<std::tuple_size_v<Args>>{});
  return 0;
}

// The runtime finds kernels by C symbol name, and a template can't have C
// language linkage, so every kernel needs an exported shim.
#define EXPORT_KERNEL(k)                                                  \
  extern "C" uint32_t k##_entry(const LaunchInfo* info, const ArgArray* a) { \
    return entry<&k>(info, a);                                            \
  }

EXPORT_KERNEL(vector_add)
```

1.2 · GENERATED · C++20

Generated from the signature

fn_traits recovers the parameter types.

Each slot index is precomputed: argument evaluation is unsequenced.

A template can’t have C linkage, so every kernel needs a macro shim.

device

```
@__name(t"entry_{get_linkage_name[kernel]()}")
def make_entry_wrapper[
    arg_types: TypeList[Trait=AnyType, ...],  # inferred from the signature
    //,
    kernel: def(*args: *arg_types) thin -> None,
](
    info: Pointer[LaunchInfo, MutAnyOrigin],
    args: Pointer[ArgArray, MutAnyOrigin],
) -> UInt32:
    var words = args.unsafe_bitcast[UInt64]()
    var slot = 0

    # Returns the address of kernel argument i. Called once per argument, in order.
    def arg_ptr[i: Int](out result: Pointer[arg_types[i], MutAnyOrigin]) {
        mut slot, imm words
    }:
        comptime T = arg_types[i]
        comptime match arg_kind[T]():
            case .static_tensor | .pointer:
                result = rebind[Pointer[T, MutAnyOrigin]](
                    words.unsafe_offset(slot + 1)
                )
                slot += 1
            case .dynamic_tensor:
                ...  # data slot + metadata slot; patch the pointer, see 3.2
                slot += 2
            case .scalar:
                result = Pointer[T, MutAnyOrigin](
                    unsafe_from_address=Int(words[unsafe_offset=slot + 1])
                )
                slot += 1

    call_with_dynamic_pack[kernel](arg_ptr)  # kernel(arg_ptr[0], ...)
    return 0
```

1.2 · GENERATED · MOJO

Generated, in Mojo

1. arg_types is inferred from the kernel's signature. C++ needs fn_traits.
2. @__name names the export after the kernel. C++ needs a macro per kernel.
3. A running slot counter, captured mutably and bumped per case. C++ precomputes each index.
4. comptime match: per-type dispatch at compile time. Problem 2 reuses arg_kind.
5. Calls the kernel with arg_ptr[0], arg_ptr[1], and so on.

Declared and compiled as on a GPU

device

```
def scale[layout: TensorLayout](
    input: TileTensor[DType.float16, layout, MutAnyOrigin],  # runtime dims are fine
    output: Pointer[Float16, MutAnyOrigin],
    factor: Float16,                                         # scalars pass by value
):
    ...
```

host

```
var input_tile = TileTensor(input_buf, row_major(Int(M), Int(N)))
var kernel = ctx.compile_function[scale[input_tile.LayoutType]]()
```

Through the compile_function_wrapper seam.

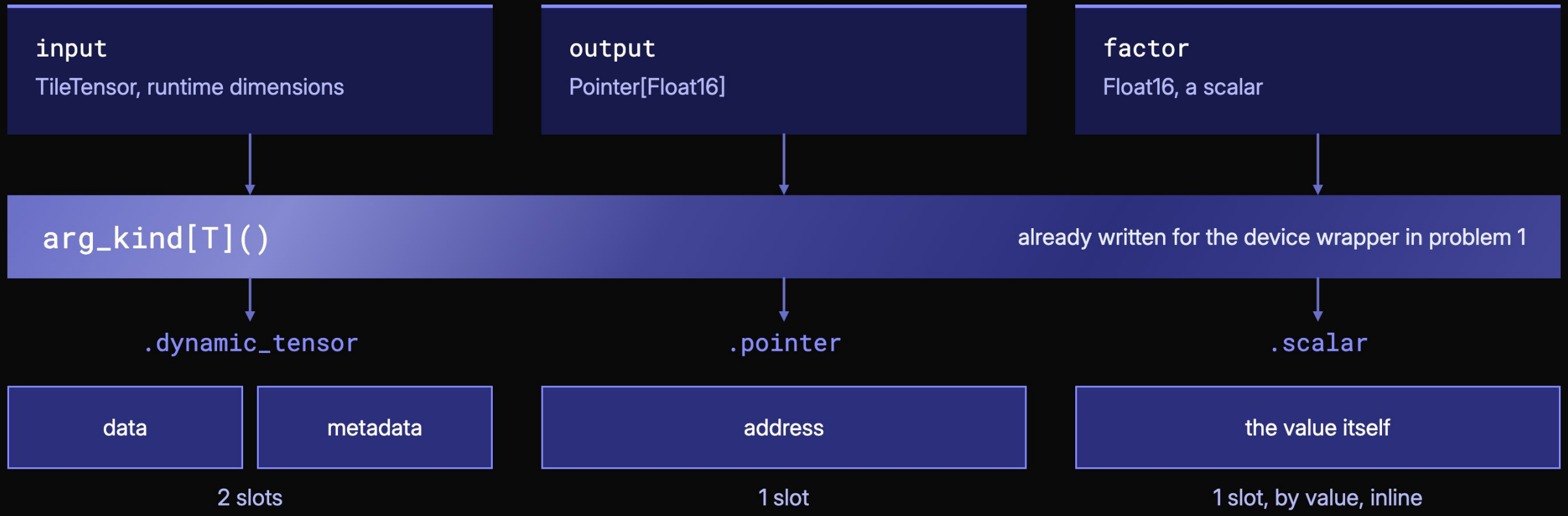
Runtime tensor dimensions are fine.

Scalars pass by value.

FROM PROBLEM 1 TO PROBLEM 2

The device side is done.

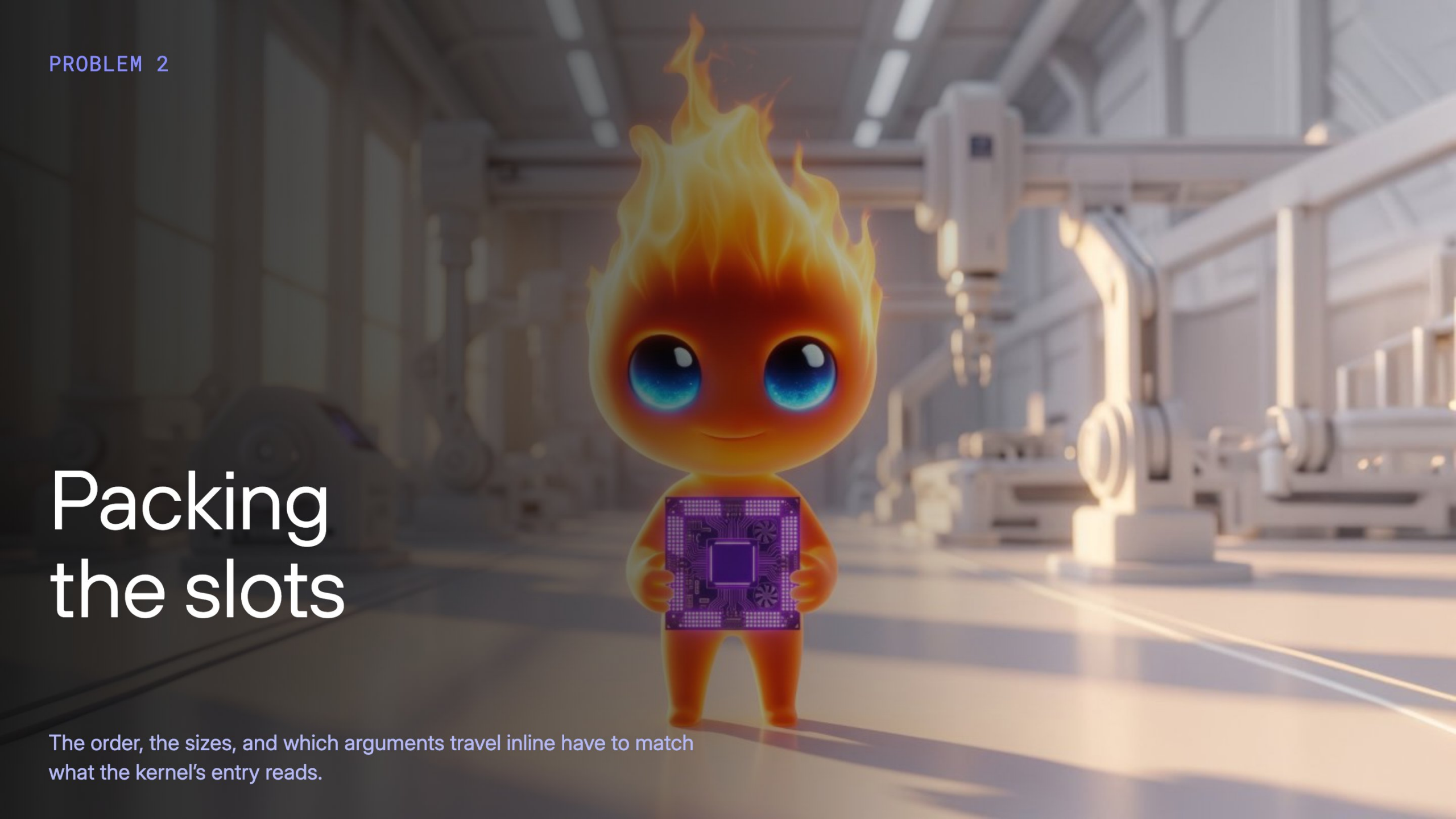
Something still has to fill the slots on the host.



Problem 2 asks the same question on the host.

PROBLEM 2

Packing the slots



The order, the sizes, and which arguments travel inline have to match what the kernel's entry reads.

Parallel arrays, at every call site

C++

```cpp
void* addrs[]     = {&lhs_buf, &rhs_buf, &out_buf, &n};
uint64_t sizes[]  = {sizeof lhs_buf, sizeof rhs_buf, sizeof out_buf, sizeof n};
bool is_inline[]  = {false, false, false, true};   // n's bytes ride with the launch
launch(kernel, addrs, sizes, is_inline, 4, grid, block);
```

Mojo

host

```mojo
var addrs = Array[OpaquePointer[MutAnyOrigin], 4](
    address_of(lhs_buf), address_of(rhs_buf), address_of(out_buf), address_of(n)
)
var sizes = Array[UInt64, 4](8, 8, 8, 4)
var is_inline = Array[Bool, 4](False, False, False, True)
ctx.launch(kernel, addrs, sizes, is_inline, 4, grid, block)
```

The slot order, the sizes, and which arguments are inline restate the kernel's slot indices from 1.1.
Nothing checks that they agree.

Classify each type, then pack and launch

```cpp
enum class ArgKind { static_tensor, dynamic_tensor, pointer, scalar };

template <class T>
constexpr ArgKind arg_kind() {
  if constexpr (is_tile_tensor_v<T> && sizeof(T) == sizeof(void*)) return ArgKind::static_tensor;
  else if constexpr (is_tile_tensor_v<T>) return ArgKind::dynamic_tensor;
  else if constexpr (std::is_pointer_v<T>) return ArgKind::pointer;
  else return ArgKind::scalar;
}

template <class T>
constexpr std::size_t slot_count() { return arg_kind<T>() == ArgKind::dynamic_tensor ? 2 : 1; }

template <class Args>
constexpr std::size_t total_slots = []<std::size_t... I>(std::index_sequence<I...>) {
  return (std::size_t{0} + ... + slot_count<std::tuple_element_t<I, Args>>());
}(std::make_index_sequence<std::tuple_size_v<Args>>{});

template <auto Kernel, class... Hs>
void pack_and_launch(Hs&... host_args) {
  using Args = typename fn_traits<decltype(Kernel)>::args;
  constexpr std::size_t n = total_slots<Args>;
  std::array<void*, n> addrs{};
  std::array<uint64_t, n> sizes{};
  std::array<bool, n> is_inline{};
  std::size_t s = 0;
  auto refs = std::forward_as_tuple(host_args...);  // to pair values with Args by index
  [&]<std::size_t... I>(std::index_sequence<I...>) {
    (pack_one<std::tuple_element_t<I, Args>>(std::get<I>(refs), addrs, sizes, is_inline, s), ...);
  }(std::index_sequence_for<Hs...>{});
  launch(addrs.data(), sizes.data(), is_inline.data(), s);
}
// pack_one<Param>(...) is an if-constexpr ladder over arg_kind<Param>(), one
// branch per case.
```

host

```mojo
def pack_and_launch[*Ts: AnyType](
    ctx: DeviceContext, f: DeviceFunction, *args: *Ts, grid_dim: Dim, block_dim: Dim
) raises:
    comptime n = total_slots[f.declared_arg_types]()
    var addrs = Array[OpaquePointer[MutAnyOrigin], n](uninitialized=True)
    var sizes = Array[UInt64, n](uninitialized=True)
    var is_inline = Array[Bool, n](fill=False)
    var slot = 0

    comptime for i in range(Ts.length):
        comptime match arg_kind[f.declared_arg_types[i]]():
            case .static_tensor | .pointer:
                addrs[slot] = address_of(args[i])
                sizes[slot] = UInt64(size_of[Ts[i]]())
                slot += 1
            case .dynamic_tensor:
                addrs[slot] = address_of(args[i])
                sizes[slot] = UInt64(size_of[Pointer[NoneType, MutAnyOrigin]]())
                addrs[slot + 1] = serialize_for[ctx.target](args[i])  # TODO, 3.2
                sizes[slot + 1] = UInt64(size_of[Ts[i], target=ctx.target]())
                is_inline[slot + 1] = True
                slot += 2
            case .scalar:
                addrs[slot] = address_of(args[i])
                sizes[slot] = UInt64(size_of[Ts[i]]())
                is_inline[slot] = True
                slot += 1

    ctx.launch(f, addrs, sizes, is_inline, slot, grid_dim, block_dim)
```

2.2 · GENERATED · MOJO

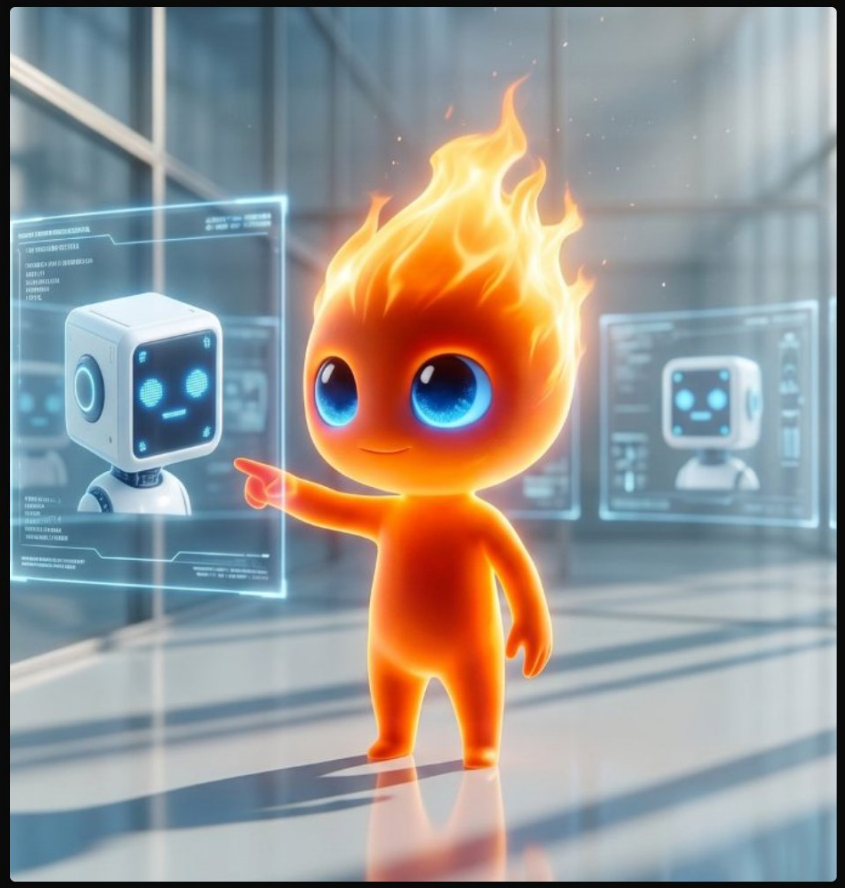
Generated, in Mojo

1. comptime for over the pack by index, with args[i] and Ts[i]. Like a list.
2. The same classification the device wrapper used. Host and device can't disagree.
3. The compiled function carries the kernel's declared parameter types.
4. The slot count is a compile-time constant: fixed-size arrays.
5. A size as the device computes it, asked from host code.

Pass values.
The classification does the rest.

```host
ctx.enqueue_function(kernel, input_tile, output_buf, Float16(0.5),
                     grid_dim=grid, block_dim=block)
```

Compare the parallel arrays in 2.1. input_tile has runtime dimensions and the scalar passes by value. Calling code remains simple.



THE SAME KERNEL, AS A CLOSURE

One kernel, no argument list

host

```
def main():
    var ctx = DeviceContext()
    var lhs = ctx.enqueue_create_buffer[DType.float32](n)
    ...                                  # rhs, result likewise

    def vector_add() {var lhs, var rhs, var result, var n}:
        var i = global_idx.x
        if i < n:
            result[unsafe_offset=i] = lhs[unsafe_offset=i] + rhs[unsafe_offset=i]

    ctx.enqueue_function(
        vector_add, grid_dim=ceildiv(n, 256), block_dim=256,
    )
    ctx.synchronize()
```

device

No argument list

Captures travel by name. Nothing to restate at the call site, and nothing to get out of sync.

Any capture works

Pointers, scalars and runtime-shaped tensors are laid out for the device, with pointers patched in.

Fusion composes

MAX's fused epilogues are closures too. The walker reaches every pointer they capture.

PROBLEM 3

Closures and pointer patching



A struct that crosses the boundary has a different layout on each side.

FROM PROBLEM 2 TO PROBLEM 3

A closure is a struct, just like a tensor's metadata

a dynamic tensor argument

```
tile: TileTensor[runtime dims]
```

is passed as

struct TileMeta

data	Pointer
dim0	Int
dim1	Int

same layout

a closure

```
def body() {var ptr, var tile, var offset, var multiplier}
```

is compiled to

struct Captures

ptr	Pointer
offset	Float32
tile: TileMeta	the same struct, nested
data	Pointer
dim0	Int
dim1	Int
multiplier	Int32

- pointer: zeroed, patched from its own slot
- Int: narrowed to 32 bits
- same size: copied

serialize_for walks any struct — so tensors and closures take the same path

```
host
def main():
    var ctx = DeviceContext()
    var ptr = ...                                  # device pointer
    var tile = ...                                 # TileTensor, runtime dims
    var offset: Float32 = 1.5
    var multiplier: Int = 4

    device
    def body() {var ptr, var tile, var offset, var multiplier}:
        var i = global_idx.x
        ptr[unsafe_offset=i] = tile[i] * Float32(multiplier) + offset

    ctx.enqueue_function(
        body, grid_dim=ceildiv(n, 256), block_dim=256,
    )
```

3.1 · MOJO

The kernel is a closure

1. Host locals that the device code uses.
2. The device code is a def with no parameters. Its capture clause names those locals.
3. The closure is enqueued like any kernel. There is no argument list to write.

WHAT THE LAUNCH HAS TO SEND

ptr	offset	tile	multiplier
Pointer[Float32]	Float32	TileTensor, runtime dims	Int

A closure is a struct of its captures. That struct has to cross to the device.

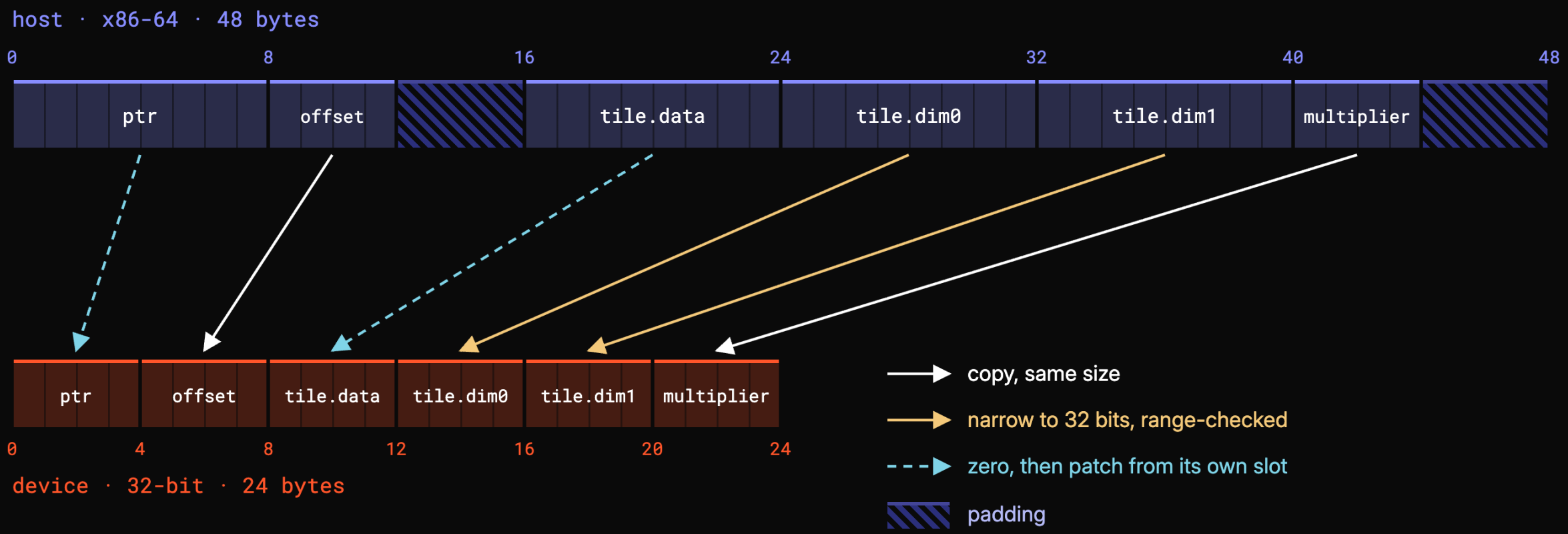
No closure. Serialize it yourself.

```
struct Tile     { float* data; ptrdiff_t dim0, dim1; };
struct Captures { float* ptr; float offset; Tile tile; int32_t multiplier; };
// sizeof(Captures): 48 on the host, 24 on the device.

// Host: device layout, kept in sync by hand.
//   ptr @0 (patched)  offset @4  tile.data @8 (patched)  dim0 @12  dim1 @16  multiplier @20
unsigned char blob[24] = {};
std::memcpy(&blob[4], &c.offset, 4);
int32_t d0 = int32_t(c.tile.dim0), d1 = int32_t(c.tile.dim1);
std::memcpy(&blob[12], &d0, 4);
std::memcpy(&blob[16], &d1, 4);
std::memcpy(&blob[20], &c.multiplier, 4);
void* addrs[] = {&grid, blob, &c.ptr, &c.tile.data};  // pointers get their own slots

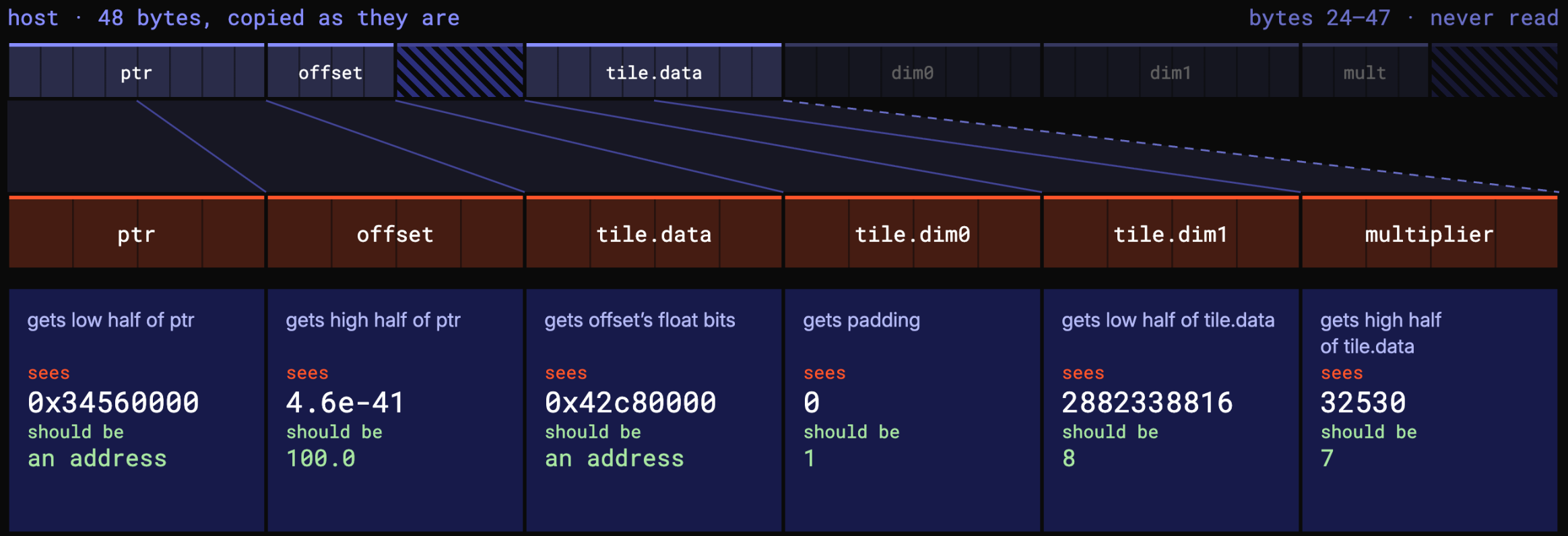
// Device: the same struct, compiled for 32 bits, then patched.
extern "C" uint32_t kernel_entry(const LaunchInfo*, const ArgArray* args) {
  Captures c;
  std::memcpy(&c, slot<unsigned char>(args, 1), sizeof c);
  c.ptr = slot<float>(args, 2);
  c.tile.data = slot<float>(args, 3);
  body(c);
  return 0;
}
```

Captures, compiled twice



Host and device lay out the same struct differently: every field after the first moves.

Copy the host's bytes, read the first 24



No crash, every number wrong.

Example values: ptr = 0x00007f1234560000, offset = 100.0, tile.data = 0x00007f12abcd0000, dim0 = 1, dim1 = 8, multiplier = 7.

PROBLEM 3 · WHAT THE LAUNCH HAS TO DO

Three steps to get a struct across from a 64-bit host to a 32-bit device

1 · RE-LAY THE STRUCT

Two layouts

host · 48 bytes



device · 24 bytes



pointer · narrowed Int · copied · padding

Every field after the first moves.

step 1

2 · PACK THE LAUNCH

A slot per pointer

ArgArray

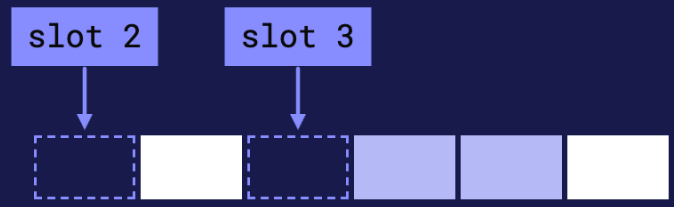


The blob is the struct in the device's layout, pointers zeroed. Each pointer gets its own slot, which the runtime fills with that buffer's device address.

step 2

3 · PATCH ON THE DEVICE

Copy, patch, call



Copy slot 1 into place, write each pointer slot back at its device offset, then call the closure.

steps 3 and 4

The next four slides take each step in turn. One walker over the struct type drives all three, compiled for both targets.

3.2 · POINTER PATCHING

Three frames, one slot per pointer



The runtime only maps slots, so every pointer gets one.

host

```mojo
def serialize_struct[T: AnyType, target: CompilationTarget](
    src: Pointer[UInt8, ImmutAnyOrigin],  # host bytes of a T
    dst: Pointer[UInt8, MutAnyOrigin],    # zeroed, size_of[T, target=target]()
) raises:
    comptime r = reflect[T]
    comptime if not r.is_struct():
        return
    comptime for i in range(r.field_count()):
        comptime F = r.field_types()[i]
        comptime host_off = r.field_offset[index=i]()
        comptime dev_off = r.field_offset[index=i, target=target]()

        comptime if size_of[F, target=target]() == size_of[F]():
            memcpy(dest=dst.unsafe_offset(dev_off),
                   src=src.unsafe_offset(host_off), count=size_of[F]())
        elif reflect[F].base_name() == "Pointer":
            pass  # left zero; patched on the device from the pointer's own slot
        elif F == Int:
            var v = src.unsafe_offset(host_off).unsafe_bitcast[Int]()[]
            if v != Int(Int32(v)):
                raise "captured Int doesn't fit in Int32"
            dst.unsafe_offset(dev_off).unsafe_bitcast[Int32]()[] = Int32(v)
        elif reflect[F].is_struct():
            serialize_struct[F, target](src.unsafe_offset(host_off),
                                        dst.unsafe_offset(dev_off))
        else:
            comptime assert False, String(
                t"capture `{reflect[F].base_name()}` differs in size on host"
                t" and device; capture it as a Pointer"
            )
```

3.2 · GENERATED · MOJO

serialize_struct

1. The same field's offset on two targets, asked from host code. The line C++ can't write.
2. Reflection over the closure's captures.
3. Copy same-size fields, leave pointers zero, narrow Int with a range check, recurse into nested structs.
4. Any other capture is a compile error that names the type.

host + device

```
trait LeafVisitor(Copyable, Movable):
    def visit_leaf(mut self, offset: Int):
        ...


def walk_pointer_leaves[T: AnyType, V: LeafVisitor](mut visitor: V, base: Int):
    comptime r = reflect[T]
    comptime if not r.is_struct():
        return
    comptime for i in range(r.field_count()):
        comptime F = r.field_types()[i]
        var offset = base + r.field_offset[index=i]()
        comptime if is_pointer_leaf[F]():  # Pointer or TileTensor
            visitor.visit_leaf(offset)
        elif reflect[F].is_struct():       # plain structs and nested closures
            walk_pointer_leaves[F, V](visitor, offset)
```

host

```
# Host, at compile time: size the slot array by running the walker.
comptime num_slots = RESERVED_SLOTS + count_leaves[ClosureType]()

# Host, at launch: FillSlots writes one slot per pointer leaf.
# Device, at entry: PatchPointers writes each slot's address back into the blob.
```

3.2 · POINTER PATCHING

One walker, three visitors

1. No target: host offsets in host code, device offsets in device code, from one function.
2. A pointer leaf is visited; a struct, nested closures included, is walked recursively.
3. The walker runs at compile time to size the slot array.

make_closure_entry

device

```
1 def make_closure_entry[C: RegisterPassable & def() -> None](
2     info: Pointer[LaunchInfo, MutAnyOrigin],
3     args: Pointer[ArgArray, MutAnyOrigin],
4 ) -> UInt32:
5     var blob = stack_allocation[size_of[C](), UInt8, alignment=align_of[C]()]()
6     memcpy(dest=blob, src=get_slot[UInt8](args, 1), count=size_of[C]())
7     var patcher = PatchPointers(blob, args.unsafe_bitcast[UInt64](), slot=2)
8     walk_pointer_leaves[C, PatchPointers](patcher, 0)
9     blob.unsafe_bitcast[C]()[]()  # call the closure
10    return 0
```

1. Copy the blob out of slot 1.
2. The walker, with the patching visitor, starting at slot 2.
3. Call the closure.
4. Compiled for the device, so these are the device's numbers.

Both capabilities are built in

01

Reflect over a closure's captures

A closure is a struct whose fields are its captures.

```
reflect[T].field_types()
```

02

Ask for the device's layout, from host code

field_offset and size_of take a target.

```
field_offset[index=i, target=t]()
```

Capture what you need. No slot indices anywhere.

host

```
var ptr = ptr_buf.unsafe_ptr()
var offset: Float32 = 100.0
var tile = TileTensor(tile_buf, row_major[1, N]())
var multiplier: Int32 = 7

def kernel() {var ptr, var offset, var tile, var multiplier}:
    for i in range(N):
        ptr[unsafe_offset=i] = ptr[unsafe_offset=i] + offset
        tile[0, i] = tile[0, i] * Float32(multiplier)

var f = ctx.compile_function_closure[type_of(kernel)]()
ctx.enqueue_function(f, kernel, grid_dim=grid, block_dim=block)
```

device

These mirror the hand-written Captures struct in 3.1, except this tile's shape is known at compile time.

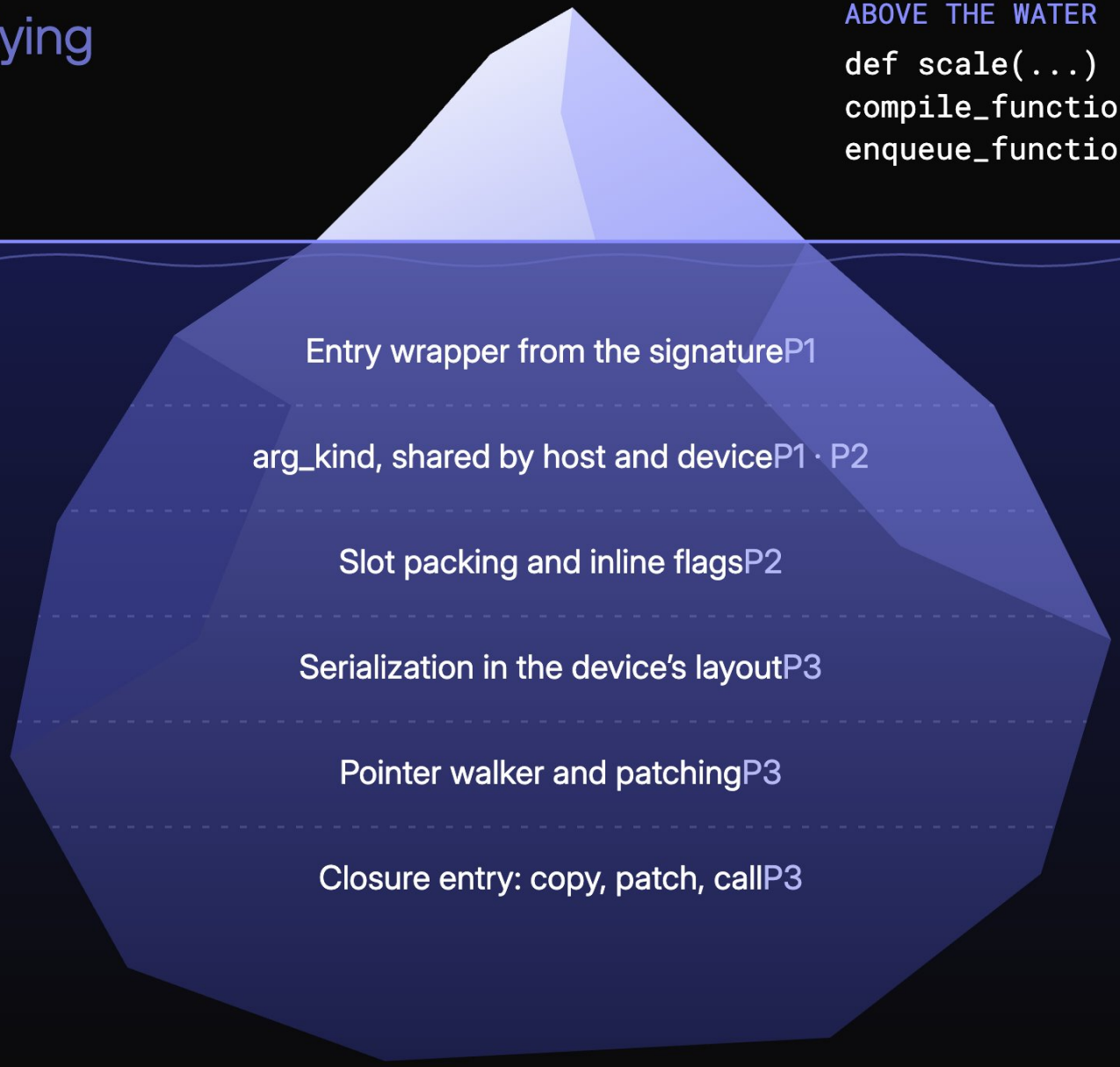
Fused epilogues arrive the same way: the walker descends to the pointers.

WHAT MOJO HIDES

Kernel authors can focus on the kernel. The mojo stdlib handles the underlying complexity.

ABOVE THE WATER

```
def scale(...)
compile_function[...]()
enqueue_function(...)
```



BELOW THE WATER

About 380 lines on the device and 520 on the host, written once in the library. None of it is in the kernel.

What they write is the GPU-shaped code they already know.

SCORECARD

C++20 gets most of the way.
Still missing some critical functionality.

Capability	Problem	C++20	Mojo
A function's parameter types	1	Traits + partial specialization	Inferred TypeList in the signature
Iterate a pack by index	1, 2	index_sequence + lambda	comptime for i in range(n), Ts[i]
Per-type dispatch	1, 2	if constexpr	comptime if / comptime match
Name a symbol after a template argument	1	Macro	@__name(t"...")
Walk a struct's fields and offsets	3	Libraries or macros	reflect[T].field_*
Walk a lambda's captures	3	Compiler support	A closure is a struct
Offset or size for another target	2, 3	No	field_offset[..., target=t], size_of[T, target=t]

The first five rows are ceremony: C++ gets there with more scaffolding. The last two are what this hardware needs, and C++ can't express them in the language.

In the compiler

Model	How it gets the captures	Layout assumption
DPC++ (Intel SYCL)	The device front end emits an integration header: one descriptor per decomposed capture, with offsets from getFieldOffset.	SPIR copies the host's widths and alignments
Upstream Clang SYCL	sycl_kernel_entry_point hands the whole kernel object to a library template, which decides how to copy it.	Copies the object as is
CUDA and HIP (clang)	A host stub builds void* args[] and calls the launch API. The driver reads parameter offsets from the kernel image.	NVPTX and AMDGPU copy the host's layout
nvcc extended lambdas	The host compiler sees a placeholder type instead of the lambda.	Captures are by value only

DPC++'s design doc decomposes captures because "there is no way in OpenCL to pass structures with pointers inside as kernel arguments".
DPC++ solves it in the compiler; Mojo solves it in a library.

Every one of these makes device layout equal host layout.

With a 32-bit device behind a 64-bit host, that option is gone.

> C++ can’t ask for the other layout: **sizeof** and **offsetof** answer only for the target being compiled.

Mojo🔥

Powerful metaprogramming.
Simple kernels.

Questions?

Modular